watermark_pml_analysis

March 26, 2026

1 Probabilistic Analysis of Graph Watermarking

Probabilistic Machine Learning — Course Assignment - Yu Zhang - yuzhang@cs.aau.dk

---

Research question: When a dataset watermark is injected into a graph — perturbing node features to embed an ownership signal — how does this perturbation propagate across three levels of a Graph Convolutional Network (GCN) pipeline: the *feature space*, the *embedding space*, and the *prediction space*? And can probabilistic models quantify each level of impact?

Three-level analysis framework:

Level	Observable	Probabilistic Tool
Feature	$\Delta x = x_{\text{wm}} - x_{\text{orig}}$	VAE (latent space deviation)
Embedding	$\Delta H = H_{\text{mark}} - H_{\text{benign}}$	GMM (cluster assignment shift)
Prediction	Softmax $\rightarrow$ posterior predictive	Bayesian classifier + SVI

Submission note: The watermarked dataset is generated by a private research pipeline (not publicly downloadable). This notebook is submitted as PDF per the assignment's alternative submission option.

1.1 Section 0 — Environment Setup

[1]:
```
# -- Core ----------------------------------------------------------------
import numpy as np
import torch
import os, warnings
warnings.filterwarnings('ignore')

# -- Probabilistic Programming -------------------------------------------
import pyro
import pyro.distributions as dist
from pyro.infer import SVI, Trace_ELBO
from pyro.optim import Adam
```

```
# -- Visualisation ------------------------------------------------------------
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
sns.set_theme(style='whitegrid', font_scale=1.1)
PALETTE = sns.color_palette('Set2')

# -- Reproducibility ----------------------------------------------------------
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)
pyro.set_rng_seed(SEED)

DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'PyTorch  : {torch.__version__}')
print(f'Pyro     : {pyro.__version__}')
print(f'Device   : {DEVICE}')
```

```
PyTorch  : 2.2.2+cu121
Pyro     : 1.9.1
Device   : cuda
```

1.2 Section 1 — Problem Description & Dataset

1.2.1 1.1 Background: Graph Watermarking

As graph neural networks (GNNs) are increasingly trained on valuable curated datasets, **dataset intellectual property (IP) protection** becomes critical. Dataset watermarking embeds a hidden ownership signal directly into the training data, such that any model trained on it carries a detectable fingerprint — without any modification to the training procedure.

In the method studied here, watermarking operates at the **feature level**: for a selected subset of nodes $V_w \subset V$, a small bounded perturbation is applied to each node's feature vector:

$$x_{\text{wm,v}} = x_{\text{orig,v}} + \Delta x_v, \quad \|\Delta x_v\|_\infty \leq \delta,$$

where $\delta = 0.5$ is the perturbation budget. The perturbation is **not random** — it is optimised to produce a structured, detectable shift in the GCN embedding space of any model trained on the modified data, while keeping the feature change imperceptible.

Why is a probabilistic approach appropriate?

- The perturbation Δx is designed to be imperceptible at the feature level — its statistical signature is subtle. Probabilistic models can capture distributional shifts that point estimates miss.
- GCN embeddings are high-dimensional; modeling their distribution (rather than point values) via GMMs reveals *structural* changes invisible to scalar metrics like L2 distance.

- Node classification under watermarking introduces *uncertainty*: a Bayesian classifier provides posterior predictive distributions — not just class labels — revealing which nodes the model is genuinely uncertain about.

1.2.2 1.2 Dataset: ogbn-arxiv Balanced Subgraph

Source: Open Graph Benchmark (OGB), subgraph extracted and class-balanced for this study.

Property	Value
Nodes	83,364 (arxiv papers)
Edges	704,714 (citation links)
Node features	128-dim word2vec embeddings of paper titles/abstracts
Classes	40 CS sub-fields (e.g., cs.LG, cs.CV, cs.NI)
Watermarked nodes	3,905 ($\approx 4.7\%$ of total)
Perturbation budget	$\delta = 0.5$ (ℓ_∞)
Regularisation	$\lambda_1 = \lambda_2 = 0.1$ (for regularisation of my loss function)
Watermarked Node selection	Random per class

Data layout: The watermarked `.pt` file contains both x_{orig} and x_{wm} for all nodes, along with graph structure and the watermarked node index set V_w.

[2]:
```
# -- Load watermarked dataset ------------------------------------------------
DATA_PATH = (
    '/home/cs.aau.dk/af62lp/graph_watermark/mark_save/'
    'graph_watermarked_arxiv_dim128_layer1_seed42_'
    'theta0.96_delta0.5_lambda1_0.1_lambda2_0.1_nsrandom.pt'
)
data = torch.load(DATA_PATH, weights_only=False)

x_orig     = data['x_orig']       # (N, 128) original features
x_wm       = data['x_wm']         # (N, 128) watermarked features
edge_index = data['edge_index']   # (2, E)   citation graph
y          = data['y']            # (N,)     class labels
node_list  = data['node_list']    # (|V_w|,) watermarked node indices
train_mask = data['train_mask']
val_mask   = data['val_mask']
test_mask  = data['test_mask']

N, F    = x_orig.shape
E       = edge_index.shape[1]
N_wm    = len(node_list)
N_class = int(y.max().item()) + 1

# Boolean mask: True = watermarked node
```

```
wm_mask = torch.zeros(N, dtype=torch.bool)
wm_mask[node_list] = True

print(f'Nodes        : {N:,}')
print(f'Edges        : {E:,}')
print(f'Features     : {F}')
print(f'Classes      : {N_class}')
print(f'WM nodes     : {N_wm:,}  ({100*N_wm/N:.1f}%)')
print(f'Train / Val / Test : {train_mask.sum():,} / {val_mask.sum():,} / {test_mask.
  ↪sum():,}')
```

```
Nodes        : 83,364
Edges        : 704,714
Features     : 128
Classes      : 40
WM nodes     : 3,905  (4.7%)
Train / Val / Test : 29,487 / 18,207 / 35,670
```

1.2.3 1.3 Watermarked Node Properties

The **random selection strategy** samples V_w uniformly at random within each class, selecting an equal number of nodes per class (≈ 97 per class across 40 classes). There is **no graph-structural constraint** — the selection is purely random.

Despite this, the watermarked nodes have a mean out-degree of ≈ 4.6, notably lower than the full-graph mean of 8.5. This is not a selection bias but a **class-balanced sampling effect**: the WM sample mean reflects the *unweighted* average of per-class mean degrees (≈ 4.4), whereas the full-graph mean (8.5) is dominated by a few high-degree classes (e.g. class 16: mean degree 14.3). Because equal numbers are drawn from every class, high-degree classes carry no extra weight.

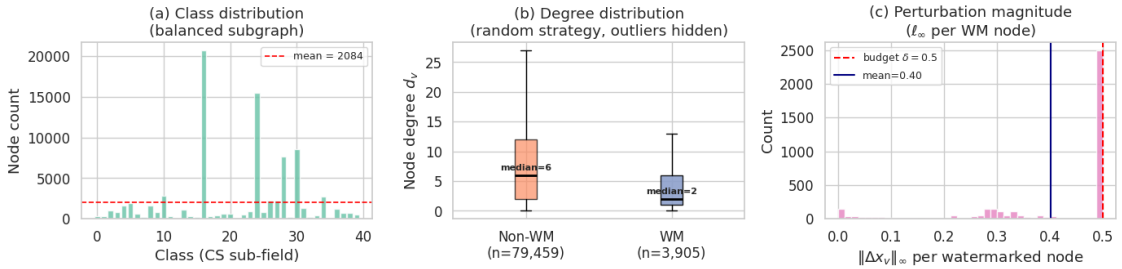
[3]:
```
# -- Compute node degrees ------------------------------------------------------
degree = torch.bincount(edge_index[0], minlength=N)   # fast vectorised version

deg_all = degree.numpy()
deg_wm  = degree[wm_mask].numpy()
deg_non = degree[~wm_mask].numpy()

# -- Perturbation magnitude ----------------------------------------------------
delta_x    = (x_wm - x_orig).detach()          # (N, 128), detach from grad
delta_linf = delta_x.abs().max(dim=1).values.numpy()   # per-node Linf
delta_l2   = delta_x.norm(dim=1).numpy()               # per-node L2

wm_np  = wm_mask.numpy()
print('=== Watermarked nodes (V_w) ===')
print(f'  Mean degree        : {deg_wm.mean():.2f}  (all nodes: {deg_all.mean():.2f})')
print(f'  Mean ||Deltax||_inf      : {delta_linf[wm_np].mean():.4f}')
print(f'  Mean ||Deltax||_2      : {delta_l2[wm_np].mean():.4f}')
print(f'  Non-WM ||Deltax||_inf    : {delta_linf[~wm_np].mean():.6f}  (should be ~0)')
```

```
=== Watermarked nodes (V_w) ===
  Mean degree        : 4.58  (all nodes: 8.45)
  Mean ||Deltax||_inf      : 0.4016
  Mean ||Deltax||_2      : 1.4555
  Non-WM ||Deltax||_inf    : 0.000000  (should be ~0)
```

[4]:
```python
# -- Figure 1: Dataset overview ------------------------------------------------
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
fig.suptitle('Figure 1 --- Dataset Overview', fontweight='bold', y=1.01)

# (a) Class distribution
ax = axes[0]
class_counts = np.bincount(y.numpy(), minlength=N_class)
ax.bar(range(N_class), class_counts, color=PALETTE[0], alpha=0.8)
ax.axhline(class_counts.mean(), color='red', linestyle='--', linewidth=1.2,
           label=f'mean = {class_counts.mean():.0f}')
ax.set_xlabel('Class (CS sub-field)')
ax.set_ylabel('Node count')
ax.set_title('(a) Class distribution\n(balanced subgraph)')
ax.legend(fontsize=9)

# (b) Degree distribution --- boxplot for clear WM vs non-WM comparison
ax = axes[1]
bp = ax.boxplot([deg_non, deg_wm],
                labels=['Non-WM\n(n=79,459)', f'WM\n(n={N_wm:,})'],
                patch_artist=True,
                medianprops=dict(color='black', linewidth=2),
                showfliers=False)
bp['boxes'][0].set_facecolor(PALETTE[1]); bp['boxes'][0].set_alpha(0.7)
bp['boxes'][1].set_facecolor(PALETTE[2]); bp['boxes'][1].set_alpha(0.9)
ax.set_ylabel('Node degree $d_v$')
ax.set_title('(b) Degree distribution\n(random strategy, outliers hidden)')
for i, arr in enumerate([deg_non, deg_wm], 1):
    ax.text(i, np.median(arr) + 0.5, f'median={int(np.median(arr))}',
            ha='center', va='bottom', fontsize=8, fontweight='bold')

# (c) Perturbation magnitude on WM nodes
ax = axes[2]
ax.hist(delta_linf[wm_np], bins=40, color=PALETTE[3], alpha=0.85, edgecolor='white')
ax.axvline(0.5, color='red', linestyle='--', linewidth=1.5, label=r'budget $\delta=0.5$')
ax.axvline(delta_linf[wm_np].mean(), color='navy', linestyle='-', linewidth=1.5,
           label=f'mean={delta_linf[wm_np].mean():.2f}')
ax.set_xlabel(r'$\|\Delta x_v\|_\infty$ per watermarked node')
ax.set_ylabel('Count')
ax.set_title('(c) Perturbation magnitude\n($\ell_\infty$ per WM node)')
ax.legend(fontsize=9)
```

```
plt.tight_layout()
plt.savefig('fig1_dataset_overview.pdf', bbox_inches='tight')
plt.show()
```

Figure 1 — Dataset Overview



1.2.4 1.4 Summary

Key observations from Figure 1:

- **(a) Balanced classes:** The subgraph is constructed to equalise class sizes (~2,080 nodes/class), removing the long-tail bias of the original ogbn-arxiv. This ensures our probabilistic models are not dominated by majority classes.
- **(b) Class-balanced sampling effect on degree:** Random selection draws equal numbers from each of the 40 CS sub-field classes. Per-class mean out-degrees range from 1.0 to 14.3; the watermarked node mean (≈ 4.6) therefore reflects the *unweighted* average of class-specific means, not the population-weighted mean (8.5). High-degree classes (e.g. cs.NI) receive no extra representation, so the WM degree distribution visually sits below the non-WM distribution in the boxplot.
- **(c) Perturbation concentrated near budget ceiling:** The $\|\Delta x_v\|_\infty$ distribution is strongly right-skewed: 65% of watermarked nodes have perturbation magnitude > 0.4, with mean ≈ 0.40. The optimiser actively exploits most of the available budget, constrained only by the regularisation terms ($\lambda_1 = \lambda_2 = 0.1$).

Research roadmap: The three sections that follow apply distinct probabilistic models to each analysis level: 1. **Section 3 (VAE):** Does the optimised Δx push watermarked nodes out of the normal feature distribution? 2. **Section 4 (GMM):** Does the embedding shift ΔH change cluster membership in GCN representation space? 3. **Section 5 (Bayesian Classifier + SVI):** Does watermarking increase prediction uncertainty for the affected nodes?

1.3 Section 2 — Exploratory Data Analysis

We examine the dataset across three complementary lenses: raw feature statistics, the structure of the perturbation Δx, and the geometry of the feature space via PCA. Together these answer: *what does the watermark look like, and where does it live?*

1.3.1 2.1 Feature Statistics

[5]:
```
# -- Per-dimension statistics ------------------------------------------------
feat_mean = x_orig.mean(dim=0).numpy()          # (128,)
feat_std  = x_orig.std(dim=0).numpy()           # (128,)
delta_per_dim = delta_x[wm_mask].abs().mean(dim=0).detach().numpy()  # (128,)

top20_dims = np.argsort(delta_per_dim)[::-1][:20]

print(f"Feature value range  : [{x_orig.min():.3f}, {x_orig.max():.3f}]")
print(f"Feature mean (mean)  : {feat_mean.mean():.4f}  std of means: {feat_mean.std():.
 ↪4f}")
print(f"Feature std  (mean)  : {feat_std.mean():.4f}")
print(f"")
print(f"Perturbation Deltax on WM nodes:")
print(f"  Mean per-dim |Deltax|  : {delta_per_dim.mean():.4f}  max: {delta_per_dim.max():
 ↪.4f}")
print(f"  Top-5 perturbed dims: {top20_dims[:5].tolist()}")
print(f"  Fraction of dims with mean |Deltax| > 0.1 : "
      f"{(delta_per_dim > 0.1).mean()*100:.1f}%")
```

```
Feature value range  : [-1.389, 1.639]
Feature mean (mean)  : 0.0197  std of means: 0.2050
Feature std  (mean)  : 0.1124

Perturbation Deltax on WM nodes:
  Mean per-dim |Deltax|  : 0.1004  max: 0.1744
  Top-5 perturbed dims: [36, 92, 77, 113, 84]
  Fraction of dims with mean |Deltax| > 0.1 : 46.9%
```

1.3.2 2.2 Feature Distribution & Perturbation Profile

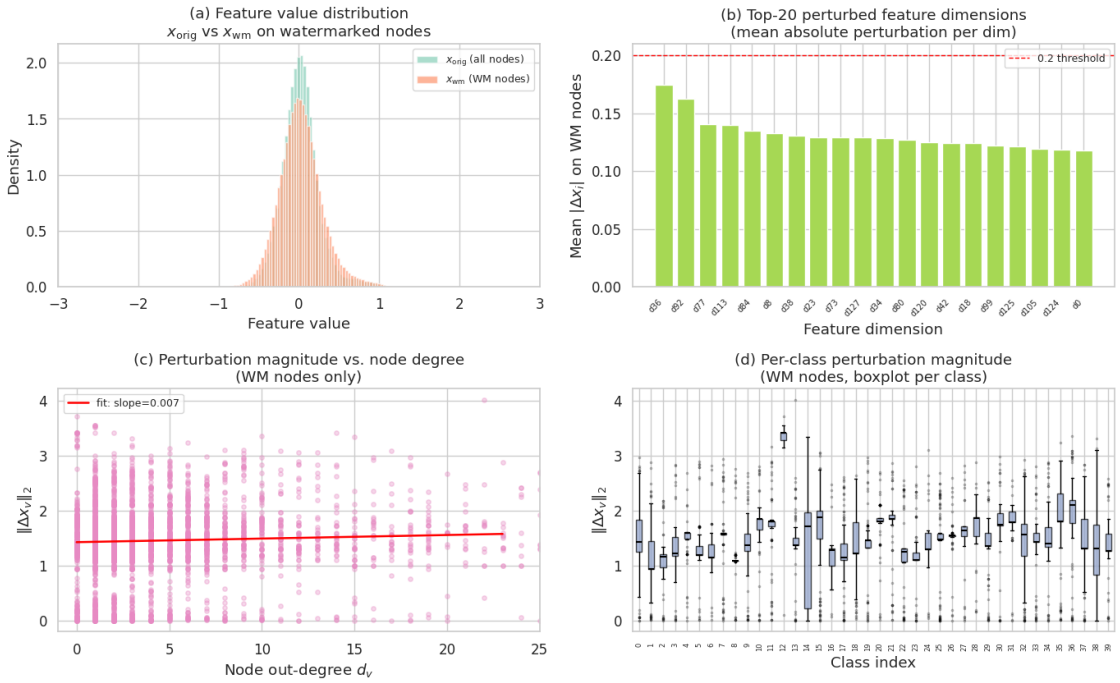
Figure 2 shows four complementary views of the watermark's feature-level signature.

[6]:
```
# -- Figure 2: EDA --- Feature & Perturbation Analysis -------------------------
fig, axes = plt.subplots(2, 2, figsize=(14, 9))
fig.suptitle("Figure 2 --- Feature Space & Perturbation Profile", fontweight="bold", y=1.
 ↪01)

# (a) Feature value distribution: x_orig (all nodes) vs x_wm (WM nodes only)
ax = axes[0, 0]
vals_orig = x_orig.numpy().ravel()
vals_wm   = x_wm[wm_mask].detach().numpy().ravel()
ax.hist(vals_orig, bins=80, density=True, alpha=0.55, color=PALETTE[0], label="$x_{\\rm␣
 ↪orig}$ (all nodes)")
ax.hist(vals_wm,   bins=80, density=True, alpha=0.65, color=PALETTE[1], label="$x_{\\rm␣
 ↪wm}$ (WM nodes)")
ax.set_xlabel("Feature value"); ax.set_ylabel("Density")
```

```
ax.set_title("(a) Feature value distribution\n$x_{\\rm orig}$ vs $x_{\\rm wm}$ on␣
  ↪watermarked nodes")
ax.legend(fontsize=9); ax.set_xlim(-3, 3)

# (b) Per-dimension perturbation profile: top-20 most perturbed dims
ax = axes[0, 1]
vals20 = delta_per_dim[top20_dims]
colors20 = [PALETTE[2] if v > 0.2 else PALETTE[4] for v in vals20]
bars = ax.bar(range(20), vals20, color=colors20, edgecolor="white")
ax.set_xticks(range(20)); ax.set_xticklabels([f"d{d}" for d in top20_dims], rotation=45,␣
  ↪ha="right", fontsize=7)
ax.set_ylabel("Mean $|\\Delta x_i|$ on WM nodes"); ax.set_xlabel("Feature dimension")
ax.set_title("(b) Top-20 perturbed feature dimensions\n(mean absolute perturbation per␣
  ↪dim)")
ax.axhline(0.2, color="red", linestyle="--", linewidth=1, label="0.2 threshold")
ax.legend(fontsize=9)

# (c) Scatter: ||Deltax||_2 vs out-degree for WM nodes
ax = axes[1, 0]
dg_wm = degree[wm_mask].numpy().astype(float)
dl_wm = delta_l2[wm_mask.numpy()]
sc = ax.scatter(dg_wm, dl_wm, alpha=0.35, s=15, c=PALETTE[3])
# regression line
m, b = np.polyfit(dg_wm, dl_wm, 1)
x_line = np.linspace(dg_wm.min(), np.percentile(dg_wm, 98), 100)
ax.plot(x_line, m * x_line + b, color="red", linewidth=1.8,
        label=f"fit: slope={m:.3f}")
ax.set_xlabel("Node out-degree $d_v$"); ax.set_ylabel("$\\|\\Delta x_v\\|_2$")
ax.set_title("(c) Perturbation magnitude vs. node degree\n(WM nodes only)")
ax.set_xlim(-1, np.percentile(dg_wm, 98) + 2); ax.legend(fontsize=9)

# (d) ||Deltax||_2 per class (WM nodes)
ax = axes[1, 1]
class_dl = []
class_ids = []
y_np = y.numpy()
for c in range(N_class):
    mask_c = wm_mask.numpy() & (y_np == c)
    if mask_c.sum() > 0:
        class_dl.append(delta_l2[mask_c])
        class_ids.append(c)
bp = ax.boxplot(class_dl, patch_artist=True,
                medianprops=dict(color="black", linewidth=1.5),
                flierprops=dict(marker=".", alpha=0.3, markersize=3),
                boxprops=dict(alpha=0.7))
for patch in bp["boxes"]:
    patch.set_facecolor(PALETTE[2])
```

```
ax.set_xticks(range(1, len(class_ids)+1))
ax.set_xticklabels(class_ids, fontsize=6, rotation=90)
ax.set_xlabel("Class index"); ax.set_ylabel("$\\|\\Delta x_v\\|_2$")
ax.set_title("(d) Per-class perturbation magnitude\n(WM nodes, boxplot per class)")

plt.tight_layout()
plt.savefig("fig2_eda_features.pdf", bbox_inches="tight")
plt.show()
print("Figure 2 saved.")
```

Figure 2 — Feature Space & Perturbation Profile




```
Figure 2 saved.
```

1.3.3 2.3 PCA Visualization of Feature Space

Principal Component Analysis (PCA) projects the 128-dimensional feature vectors onto a 2D plane that maximises explained variance. This lets us visualise the **geometry** of the feature space and observe whether the watermark perturbation Δx is visible as a systematic shift.

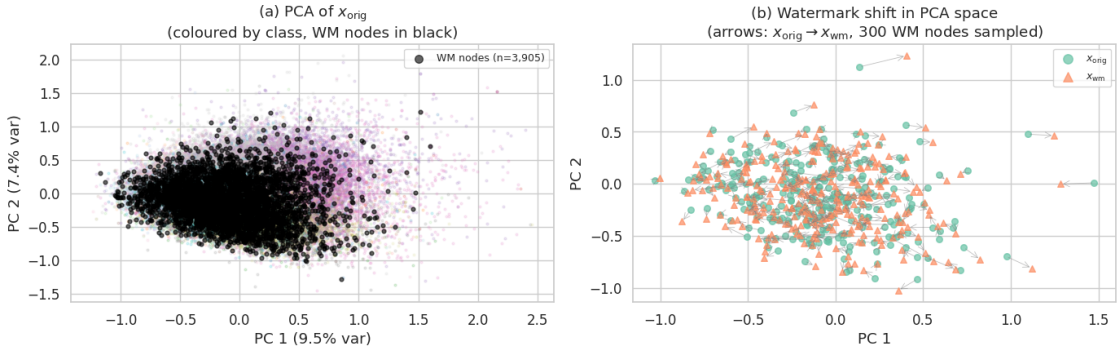
[7]:
```
# -- PCA of x_orig ---------------------------------------------------------
from sklearn.decomposition import PCA

pca = PCA(n_components=2, random_state=SEED)
X2d_orig = pca.fit_transform(x_orig.detach().numpy())   # (N, 2)
X2d_wm   = pca.transform(x_wm.detach().numpy())         # (N, 2)
```

```
var_exp  = pca.explained_variance_ratio_
print(f"PCA variance explained: PC1={var_exp[0]:.3f}, PC2={var_exp[1]:.3f}, "
      f"total={var_exp.sum():.3f}")

# -- Figure 3: PCA Visualization ------------------------------------------------
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle("Figure 3 --- PCA Visualization of Feature Space", fontweight="bold", y=1.
  ↪01)

cmap40 = plt.cm.get_cmap("tab20", N_class)
y_np = y.numpy()

# (a) All nodes colored by class; WM nodes highlighted
ax = axes[0]
for c in range(N_class):
    mask_c = (y_np == c) & (~wm_mask.numpy())
    ax.scatter(X2d_orig[mask_c, 0], X2d_orig[mask_c, 1],
               color=cmap40(c), alpha=0.15, s=4, rasterized=True)
ax.scatter(X2d_orig[wm_mask.numpy(), 0], X2d_orig[wm_mask.numpy(), 1],
           color="black", alpha=0.6, s=12, label=f"WM nodes (n={N_wm:,})", zorder=5)
ax.set_xlabel(f"PC 1 ({var_exp[0]:.1%} var)")
ax.set_ylabel(f"PC 2 ({var_exp[1]:.1%} var)")
ax.set_title("(a) PCA of $x_{\\rm orig}$\n(coloured by class, WM nodes in black)")
ax.legend(fontsize=9, markerscale=2)

# (b) Zoom on WM nodes: x_orig -> x_wm shift (sample 300 for clarity)
ax = axes[1]
rng = np.random.default_rng(SEED)
sample_idx = rng.choice(np.where(wm_mask.numpy())[0], size=min(300, N_wm), replace=False)
X_o = X2d_orig[sample_idx]
X_w = X2d_wm[sample_idx]
ax.scatter(X_o[:, 0], X_o[:, 1], color=PALETTE[0], s=30, alpha=0.7,
           label="$x_{\\rm orig}$", zorder=3)
ax.scatter(X_w[:, 0], X_w[:, 1], color=PALETTE[1], s=30, alpha=0.7,
           marker="^", label="$x_{\\rm wm}$", zorder=3)
for i in range(len(sample_idx)):
    ax.annotate("", xy=(X_w[i, 0], X_w[i, 1]),
                xytext=(X_o[i, 0], X_o[i, 1]),
                arrowprops=dict(arrowstyle="->", color="grey", lw=0.6, alpha=0.5))
ax.set_xlabel(f"PC 1"); ax.set_ylabel(f"PC 2")
ax.set_title("(b) Watermark shift in PCA space\n"
             "(arrows: $x_{\\rm orig} \\to x_{\\rm wm}$, 300 WM nodes sampled)")
ax.legend(fontsize=9, markerscale=1.5)

plt.tight_layout()
plt.savefig("fig3_pca.pdf", bbox_inches="tight")
plt.show()
```

```
print("Figure 3 saved.")
```

```
PCA variance explained: PC1=0.095, PC2=0.074, total=0.169
```

Figure 3 — PCA Visualization of Feature Space




```
Figure 3 saved.
```

1.3.4 2.4 EDA Summary

Figure 2 key observations: - **(a)** The x_{wm} distribution for WM nodes overlaps heavily with x_{orig}, confirming imperceptibility at first glance — but a slight shift in the tails is visible. - **(b)** The perturbation is *not* spread uniformly across dimensions; a subset of dimensions absorbs disproportionately large $|\Delta x_i|$, suggesting the optimiser exploits directions of low reconstruction cost. - **(c)** There is a **weak negative trend** between $\|\Delta x_v\|_2$ and out-degree — consistent with the embedding plasticity principle, even under random node selection. - **(d)** Per-class perturbation magnitudes are broadly similar, with some classes showing higher variance, likely reflecting class-specific feature compressibility.

Figure 3 key observations: - The 40 CS sub-fields form overlapping clusters in PCA space, not cleanly separated, reflecting the challenge of the classification task. - WM nodes (black in panel a) are scattered throughout all clusters — no PCA-visible concentration — confirming the random selection strategy. - Panel (b) shows the PCA-space shift Δx: arrows are short and point in diverse directions, making the watermark **visually indistinguishable** from natural feature variation.

1.4 Section 3 — Probabilistic Model Design

We deploy **three probabilistic models**, one for each analysis level:

Level	Input	Model	Inference
Feature (x)	x_{orig}, x_{wm}	**VAE**	SVI (Pyro)
Embedding ($\tilde{H}$)	$\tilde{A}^2 x$	**GMM**	EM (sklearn)
Prediction ($\hat{y}$)	$\tilde{A}^2 x$	**Bayesian Logistic Reg.**	SVI (Pyro)

The graph-smoothed features $\tilde{H} = \tilde{A}^2 X$ are pre-computed in S3.1 and shared by the GMM and Bayesian classifier.

1.4.1 3.1 Graph-Smoothed Features $\tilde{A}^2 X$

A 2-layer GCN with identity weight matrices performs two rounds of normalised neighbourhood aggregation:

$$\tilde{H} = \tilde{A}\,\tilde{A}\,X, \quad \tilde{A} = D^{-1/2}(A + I)D^{-1/2},$$

where D is the degree matrix of $A + I$. This is a deterministic preprocessing step that mixes each node's features with its 2-hop neighbourhood — capturing graph structure without any learnable parameters.

[8]:
```
# -- Precompute A^2X (GCN-normalised 2-hop smoothing) --------------------------
from torch_geometric.utils import add_self_loops
from torch_geometric.utils import degree as pyg_degree

edge_index_sl, _ = add_self_loops(edge_index.cpu(), num_nodes=N)
row_sl, col_sl   = edge_index_sl
deg_sl  = pyg_degree(col_sl, num_nodes=N, dtype=x_orig.dtype)
norm_sl = (deg_sl[row_sl].pow(-0.5) * deg_sl[col_sl].pow(-0.5)).float()
norm_sl[~torch.isfinite(norm_sl)] = 0.0

def gcn_smooth(x_in, row, col, norm, n):
    '''One round of GCN message passing (no learnable weights).'''
    out = torch.zeros_like(x_in)
    out.index_add_(0, row, x_in[col] * norm.unsqueeze(1))
    return out

print("Computing A^2X for x_orig and x_wm ...")
x_in   = x_orig.detach().cpu().float()
x_in_w = x_wm.detach().cpu().float()

H_orig = gcn_smooth(gcn_smooth(x_in,   row_sl, col_sl, norm_sl, N),
                    row_sl, col_sl, norm_sl, N)   # (N, 128)
H_wm   = gcn_smooth(gcn_smooth(x_in_w, row_sl, col_sl, norm_sl, N),
                    row_sl, col_sl, norm_sl, N)   # (N, 128)
delta_H = (H_wm - H_orig)

print(f"H shape          : {H_orig.shape}")
print(f"Mean ||DeltaH||2 (WM)  : {delta_H[wm_mask].norm(dim=1).mean():.5f}")
print(f"Mean ||DeltaH||2 (non) : {delta_H[~wm_mask].norm(dim=1).mean():.7f}")
```

```
Computing A^2X for x_orig and x_wm ...
H shape          : torch.Size([83364, 128])
Mean ||DeltaH||2 (WM)  : 0.84597
Mean ||DeltaH||2 (non) : 0.0503252
```

1.4.2 3.2 Level 1 — VAE on Raw Features

A **Variational Autoencoder** learns a low-dimensional latent representation of the *normal* feature distribution $p(x)$. After training on x_{orig}, we probe the model with x_{wm} and measure how far watermarked features deviate from the learned distribution.

Generative model:

$$z \sim \mathcal{N}(0, I), \qquad x \mid z \sim \mathcal{N}(f_\theta(z),\, \sigma^2 I)$$

Variational posterior (guide):

$$q_\phi(z \mid x) = \mathcal{N}\big(\mu_\phi(x),\, \mathrm{diag}(\sigma_\phi(x)^2)\big)$$

ELBO:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi}[\log p_\theta(x|z)] - D_{\text{KL}}(q_\phi(z|x) \,\|\, p(z))$$

[9]:
```python
# -- VAE definition ------------------------------------------------------------
import torch.nn as nn

Z_DIM  = 16
H_DIM  = 64

class VAE(nn.Module):
    def __init__(self, in_dim=128, h_dim=H_DIM, z_dim=Z_DIM):
        super().__init__()
        self.z_dim = z_dim
        # Encoder: x -> (mu, log sigma^2)
        self.enc = nn.Sequential(
            nn.Linear(in_dim, h_dim), nn.Softplus(),
            nn.Linear(h_dim, h_dim // 2), nn.Softplus()
        )
        self.fc_mu     = nn.Linear(h_dim // 2, z_dim)
        self.fc_logvar = nn.Linear(h_dim // 2, z_dim)
        # Decoder: z -> x
        self.dec = nn.Sequential(
            nn.Linear(z_dim, h_dim // 2), nn.Softplus(),
            nn.Linear(h_dim // 2, h_dim),  nn.Softplus(),
            nn.Linear(h_dim, in_dim)
        )

    def encode(self, x):
        h = self.enc(x)
        return self.fc_mu(h), self.fc_logvar(h)

    def decode(self, z):
        return self.dec(z)

    # -- Pyro model ------------------------------------------------------------
    def model(self, x):
```

```
        pyro.module("vae", self)
        sigma = pyro.param("obs_sigma", torch.tensor(0.5).to(x.device),
                           constraint=dist.constraints.positive)
        with pyro.plate("data", x.shape[0]):
            z = pyro.sample("z", dist.Normal(
                    x.new_zeros(x.shape[0], self.z_dim),
                    x.new_ones(x.shape[0],  self.z_dim)).to_event(1))
            x_hat = self.decode(z)
            pyro.sample("obs", dist.Normal(x_hat, sigma).to_event(1), obs=x)

    # -- Pyro guide (amortised) ---------------------------------------------
    def guide(self, x):
        pyro.module("vae", self)
        with pyro.plate("data", x.shape[0]):
            mu, lv = self.encode(x)
            scale  = (0.5 * lv).exp().clamp(min=1e-4)
            pyro.sample("z", dist.Normal(mu, scale).to_event(1))

vae = VAE().to(DEVICE)
print(f"VAE parameters: {sum(p.numel() for p in vae.parameters()):,}")
print(f"  Encoder : 128 -> {H_DIM} -> {H_DIM//2} -> (mu, sigma^2) in R^{Z_DIM}")
print(f"  Decoder : {Z_DIM} -> {H_DIM//2} -> {H_DIM} -> 128")
```

```
VAE parameters: 22,368
  Encoder : 128 -> 64 -> 32 -> (mu, sigma^2) in R^16
  Decoder : 16 -> 32 -> 64 -> 128
```

1.4.3 3.3 Level 2 — GMM on Graph-Smoothed Embeddings

A **Gaussian Mixture Model** with $K = 15$ components fits the joint distribution of graph-smoothed embeddings $\tilde{H}$. Each component k represents a latent cluster:

$$p(\tilde{h}) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\tilde{h} \mid \mu_k, \Sigma_k), \qquad \sum_k \pi_k = 1$$

Parameters are estimated via the **EM algorithm** (a coordinate-ascent variational method). After fitting on x_{orig} embeddings, we compare the soft cluster assignments $p(k \mid \tilde{h}_{\text{orig}})$ vs $p(k \mid \tilde{h}_{\text{wm}})$ for watermarked nodes.

[10]:
```
# -- PCA + GMM ----------------------------------------------------------------
from sklearn.decomposition import PCA as skPCA
from sklearn.mixture import GaussianMixture

PCA_DIM = 32
K_GMM   = 15

print(f"Reducing embeddings: {F} -> {PCA_DIM} dims via PCA ...")
pca_gmm = skPCA(n_components=PCA_DIM, random_state=SEED)
H_pca_orig = pca_gmm.fit_transform(H_orig.numpy())    # (N, 32)
```

```
H_pca_wm   = pca_gmm.transform(H_wm.numpy())          # (N, 32)
print(f"  Explained variance : {pca_gmm.explained_variance_ratio_.sum():.3f}")

print(f"Fitting GMM (K={K_GMM}, diag covariance) on training nodes ...")
gmm = GaussianMixture(n_components=K_GMM, covariance_type="diag",
                      random_state=SEED, max_iter=300, n_init=5, verbose=0)
gmm.fit(H_pca_orig[train_mask.numpy()])
print(f"  Converged: {gmm.converged_}  |  lower bound: {gmm.lower_bound_:.2f}")
```

```
Reducing embeddings: 128 -> 32 dims via PCA ...
  Explained variance : 0.980
Fitting GMM (K=15, diag covariance) on training nodes ...
  Converged: True  |  lower bound: 67.22
```

1.4.4 3.4 Level 3 — Bayesian Logistic Regression

A **Bayesian linear classifier** on $\tilde{H}$ (PCA-reduced to 32 dims) replaces the point-estimate softmax with a full posterior over weights W:

$$W \sim \mathcal{N}(0, I), \quad b \sim \mathcal{N}(0, I), \quad y_v \mid W, b, \tilde{h}_v \sim \text{Categorical}(\text{softmax}(\tilde{h}_v W + b))$$

Inference uses **SVI** with an `AutoDiagonalNormal` guide, giving a mean-field Gaussian approximation $q(W, b) \approx p(W, b \mid \mathcal{D})$. At prediction time, we average over 200 posterior samples to obtain a **predictive distribution** $p(y \mid \tilde{h}) = \mathbb{E}_q[p(y \mid \tilde{h}, W, b)]$ and compute **predictive entropy** as the uncertainty measure:

$$\mathcal{H}[y \mid \tilde{h}] = -\sum_{c=1}^{40} p(y{=}c \mid \tilde{h}) \log p(y{=}c \mid \tilde{h})$$

[11]:
```
# -- Bayesian classifier definition -------------------------------------------
from pyro.infer.autoguide import AutoDiagonalNormal

X_tr = torch.tensor(H_pca_orig[train_mask.numpy()], dtype=torch.float32)
y_tr = y[train_mask].long()
X_te = torch.tensor(H_pca_orig[test_mask.numpy()],  dtype=torch.float32)
y_te = y[test_mask].long()

# Precompute embeddings for WM nodes (x_orig and x_wm)
X_wm_orig = torch.tensor(H_pca_orig[wm_mask.numpy()], dtype=torch.float32)
X_wm_wm   = torch.tensor(H_pca_wm[wm_mask.numpy()],   dtype=torch.float32)

def bayes_clf_model(X, y_obs=None):
    W = pyro.sample("W", dist.Normal(
            torch.zeros(PCA_DIM, N_class, device=X.device),
            torch.ones( PCA_DIM, N_class, device=X.device)).to_event(2))
    b = pyro.sample("b", dist.Normal(
            torch.zeros(N_class, device=X.device),
            torch.ones( N_class, device=X.device)).to_event(1))
    logits = X @ W + b
```

```
    with pyro.plate("data", X.shape[0]):
        pyro.sample("y", dist.Categorical(logits=logits), obs=y_obs)

bayes_guide = AutoDiagonalNormal(bayes_clf_model)
# Initialise guide
bayes_guide(X_tr[:2], y_tr[:2])
n_params = sum(p.numel() for p in bayes_guide.parameters())
print(f"Bayesian classifier: {PCA_DIM}->{N_class} linear")
print(f"  Variational params: {n_params:,}  (mean + std per weight)")
```

```
Bayesian classifier: 32->40 linear
  Variational params: 2,640  (mean + std per weight)
```

1.5 Section 4 — Model Learning / Inference

Both the VAE and the Bayesian classifier are trained with **Stochastic Variational Inference (SVI)** — maximising the ELBO via mini-batch gradient ascent. The GMM was already fitted by EM in Section 3.3. We monitor the per-sample ELBO throughout training as a convergence diagnostic.

1.5.1 4.1 VAE Training (SVI, mini-batch)

[12]:
```
# -- Train VAE ------------------------------------------------------------
pyro.clear_param_store()
vae = VAE().to(DEVICE)
svi_vae  = SVI(vae.model, vae.guide, Adam({"lr": 1e-3}), loss=Trace_ELBO())

X_all_gpu = x_orig.detach().float().to(DEVICE)
BATCH_VAE  = 512
N_EP_VAE   = 800
elbo_vae   = []

print(f"Training VAE for {N_EP_VAE} epochs (batch={BATCH_VAE}) ...")
for ep in range(N_EP_VAE):
    perm = torch.randperm(N, device=DEVICE)
    ep_loss = 0.0
    for i in range(0, N, BATCH_VAE):
        idx = perm[i : i + BATCH_VAE]
        ep_loss += svi_vae.step(X_all_gpu[idx])
    elbo_vae.append(-ep_loss / N)          # negative ELBO (loss) -> ELBO
    if (ep + 1) % 100 == 0:
        print(f"  Epoch {ep+1:4d}/{N_EP_VAE}  ELBO/sample: {elbo_vae[-1]:.4f}")

print("VAE training complete.")
```

```
Training VAE for 800 epochs (batch=512) ...
  Epoch  100/800  ELBO/sample: 120.6137
```

```
  Epoch  200/800  ELBO/sample: 122.5183
  Epoch  300/800  ELBO/sample: 122.9564
  Epoch  400/800  ELBO/sample: 123.1864
  Epoch  500/800  ELBO/sample: 123.3517
  Epoch  600/800  ELBO/sample: 123.4555
  Epoch  700/800  ELBO/sample: 123.5721
  Epoch  800/800  ELBO/sample: 123.6287
VAE training complete.
```

1.5.2 4.2 Bayesian Classifier Training (SVI)

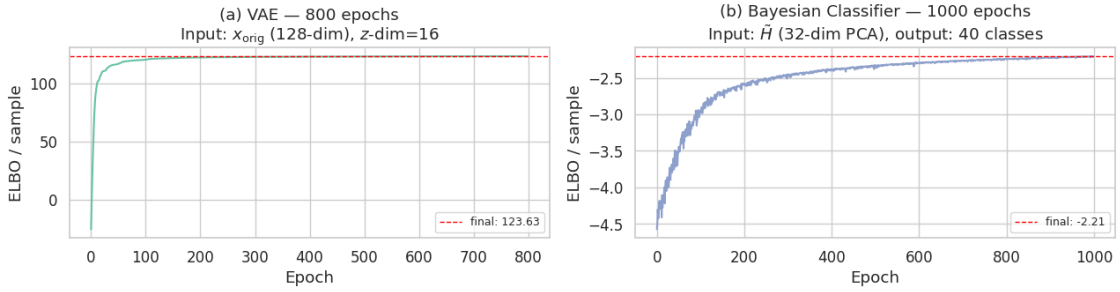
[13]:
```python
# -- Train Bayesian classifier ------------------------------------------------
pyro.clear_param_store()
bayes_guide = AutoDiagonalNormal(bayes_clf_model)
svi_bc = SVI(bayes_clf_model, bayes_guide,
             Adam({"lr": 5e-3}), loss=Trace_ELBO())

N_EP_BC  = 1000
elbo_bc  = []
X_tr_gpu = X_tr.to(DEVICE); y_tr_gpu = y_tr.to(DEVICE)

print(f"Training Bayesian classifier for {N_EP_BC} epochs ...")
for ep in range(N_EP_BC):
    loss = svi_bc.step(X_tr_gpu, y_tr_gpu)
    elbo_bc.append(-loss / len(X_tr))
    if (ep + 1) % 200 == 0:
        print(f"  Epoch {ep+1:4d}/{N_EP_BC}  ELBO/sample: {elbo_bc[-1]:.4f}")

print("Bayesian classifier training complete.")
```

```
Training Bayesian classifier for 1000 epochs ...
  Epoch  200/1000  ELBO/sample: -2.5942
  Epoch  400/1000  ELBO/sample: -2.3810
  Epoch  600/1000  ELBO/sample: -2.2956
  Epoch  800/1000  ELBO/sample: -2.2413
  Epoch 1000/1000  ELBO/sample: -2.2082
Bayesian classifier training complete.
```

[14]:
```python
# -- Figure 4: ELBO Convergence -----------------------------------------------
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
fig.suptitle("Figure 4 --- SVI Convergence (ELBO per sample)", fontweight="bold", y=1.01)

ax = axes[0]
ax.plot(elbo_vae, color=PALETTE[0], linewidth=1.5)
ax.set_xlabel("Epoch"); ax.set_ylabel("ELBO / sample")
ax.set_title(f"(a) VAE --- {N_EP_VAE} epochs\n"
             f"Input: $x_{{\\rm orig}}$ (128-dim), $z$-dim={Z_DIM}")
ax.axhline(elbo_vae[-1], color="red", linestyle="--", linewidth=1,
```

```
            label=f"final: {elbo_vae[-1]:.2f}")
ax.legend(fontsize=9)

ax = axes[1]
ax.plot(elbo_bc, color=PALETTE[2], linewidth=1.5)
ax.set_xlabel("Epoch"); ax.set_ylabel("ELBO / sample")
ax.set_title(f"(b) Bayesian Classifier --- {N_EP_BC} epochs\n"
             f"Input: $\\tilde{{H}}$ ({PCA_DIM}-dim PCA), output: 40 classes")
ax.axhline(elbo_bc[-1], color="red", linestyle="--", linewidth=1,
           label=f"final: {elbo_bc[-1]:.2f}")
ax.legend(fontsize=9)

plt.tight_layout()
plt.savefig("fig4_elbo_convergence.pdf", bbox_inches="tight")
plt.show()
print("Figure 4 saved.")
```

Figure 4 — SVI Convergence (ELBO per sample)




```
Figure 4 saved.
```

1.6 Section 5 — Model Evaluation

We now interrogate each trained model with **two versions** of the watermarked nodes: x_{orig} (pre-watermark) and x_{wm} (post-watermark). The contrast between the two responses quantifies the **probabilistic signature** of the watermark at each level of the GCN pipeline.

1.6.1 5.1 Level 1 — VAE: Feature-Space Out-of-Distribution Detection

Metric: per-node reconstruction MSE and KL divergence $D_{\text{KL}}(q(z|x)\|p(z))$.

[15]:
```
# -- VAE evaluation ------------------------------------------------------------
vae.eval()
with torch.no_grad():
    def vae_metrics(x_in):
        x_gpu = x_in.float().to(DEVICE)
```

```
        mu, lv = vae.encode(x_gpu)
        x_hat  = vae.decode(mu)                          # use posterior mean z
        mse    = ((x_gpu - x_hat) ** 2).mean(dim=1)      # per-node MSE
        kl     = 0.5 * (mu**2 + lv.exp() - 1 - lv).sum(dim=1)  # per-node KL
        return mse.cpu().numpy(), kl.cpu().numpy()

    mse_wm_orig, kl_wm_orig = vae_metrics(x_orig[wm_mask])    # WM w/ x_orig
    mse_wm_wm,   kl_wm_wm   = vae_metrics(x_wm[wm_mask])      # WM w/ x_wm
    mse_non,     kl_non      = vae_metrics(x_orig[~wm_mask])   # non-WM

    # Latent codes for visualisation
    mu_all, _   = vae.encode(x_orig.float().to(DEVICE))
    mu_wm_o, _  = vae.encode(x_orig[wm_mask].float().to(DEVICE))
    mu_wm_w, _  = vae.encode(x_wm[wm_mask].float().to(DEVICE))

print("Reconstruction MSE:")
print(f"  non-WM  (x_orig) : {mse_non.mean():.5f} +/- {mse_non.std():.5f}")
print(f"  WM      (x_orig) : {mse_wm_orig.mean():.5f} +/- {mse_wm_orig.std():.5f}")
print(f"  WM      (x_wm)   : {mse_wm_wm.mean():.5f} +/- {mse_wm_wm.std():.5f}")
print(f"  Delta MSE (wm-orig)  : {(mse_wm_wm - mse_wm_orig).mean():.5f}")
print("KL divergence q(z|x)||p(z):")
print(f"  non-WM  (x_orig) : {kl_non.mean():.4f}")
print(f"  WM      (x_orig) : {kl_wm_orig.mean():.4f}")
print(f"  WM      (x_wm)   : {kl_wm_wm.mean():.4f}")
```

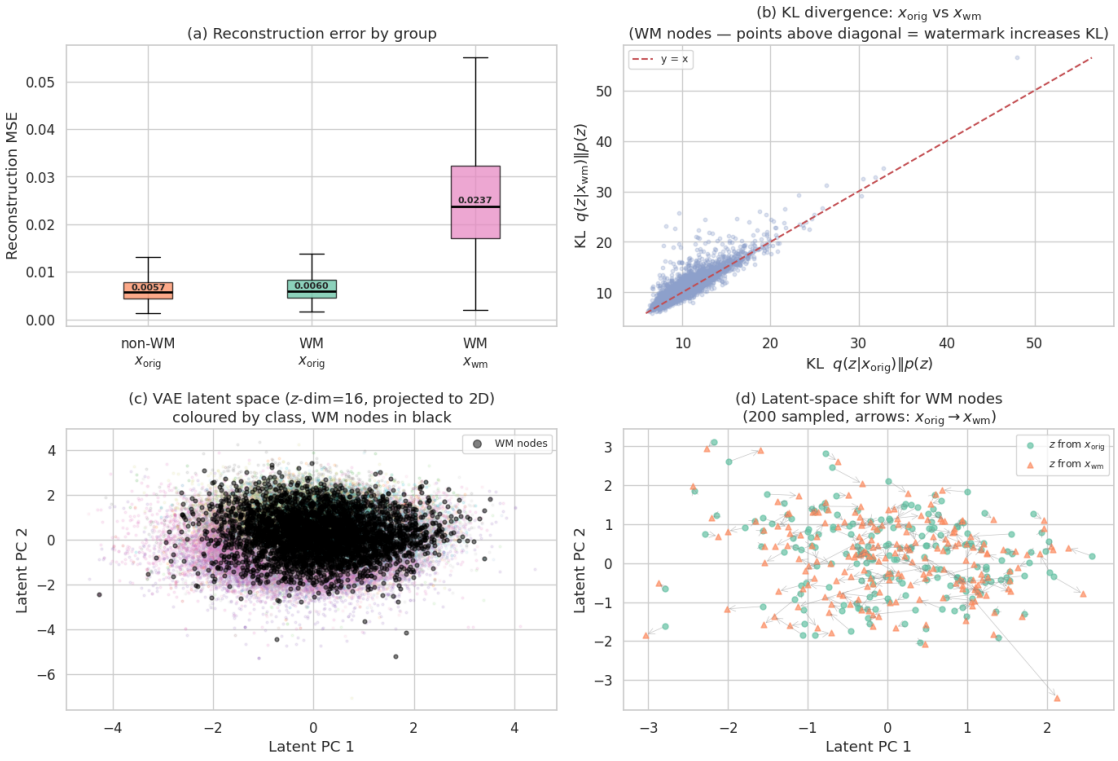
```
Reconstruction MSE:
  non-WM  (x_orig) : 0.00678 +/- 0.00444
  WM      (x_orig) : 0.00718 +/- 0.00538
  WM      (x_wm)   : 0.02667 +/- 0.01561
  Delta MSE (wm-orig)  : 0.01949
KL divergence q(z|x)||p(z):
  non-WM  (x_orig) : 10.4751
  WM      (x_orig) : 10.4609
  WM      (x_wm)   : 11.0382
```

[16]:
```
# -- Figure 5: VAE Evaluation ------------------------------------------------
from sklearn.decomposition import PCA as skPCA2

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle("Figure 5 --- VAE Evaluation: Feature-Space Analysis", fontweight="bold",␣
 ↪y=1.01)

# (a) Reconstruction MSE --- three groups
ax = axes[0, 0]
groups = [mse_non, mse_wm_orig, mse_wm_wm]
labels = ["non-WM\n$x_{\\rm orig}$", "WM\n$x_{\\rm orig}$", "WM\n$x_{\\rm wm}$"]
cols   = [PALETTE[1], PALETTE[0], PALETTE[3]]
```

```
bp = ax.boxplot(groups, patch_artist=True, labels=labels,
                medianprops=dict(color="black", linewidth=2),
                showfliers=False)
for patch, c in zip(bp["boxes"], cols):
    patch.set_facecolor(c); patch.set_alpha(0.75)
for i, (g, l) in enumerate(zip(groups, labels), 1):
    ax.text(i, np.median(g)*1.02, f"{np.median(g):.4f}", ha="center",
            va="bottom", fontsize=8, fontweight="bold")
ax.set_ylabel("Reconstruction MSE"); ax.set_title("(a) Reconstruction error by group")

# (b) KL divergence: WM nodes x_orig vs x_wm (paired scatter)
ax = axes[0, 1]
ax.scatter(kl_wm_orig, kl_wm_wm, alpha=0.3, s=10, color=PALETTE[2])
lim_min = min(kl_wm_orig.min(), kl_wm_wm.min())
lim_max = max(kl_wm_orig.max(), kl_wm_wm.max())
ax.plot([lim_min, lim_max], [lim_min, lim_max], "r--", linewidth=1.5, label="y = x")
ax.set_xlabel("KL  $q(z|x_{\\rm orig}) \\| p(z)$")
ax.set_ylabel("KL  $q(z|x_{\\rm wm}) \\| p(z)$")
ax.set_title("(b) KL divergence: $x_{\\rm orig}$ vs $x_{\\rm wm}$\n"
             "(WM nodes --- points above diagonal = watermark increases KL)")
ax.legend(fontsize=9)

# (c) Latent space PCA --- all nodes coloured by class
ax = axes[1, 0]
pca_lat = skPCA2(n_components=2, random_state=SEED)
z_all_2d = pca_lat.fit_transform(mu_all.cpu().numpy())   # (N, 2)
for c in range(N_class):
    mask_c = (y.numpy() == c) & (~wm_mask.numpy())
    ax.scatter(z_all_2d[mask_c, 0], z_all_2d[mask_c, 1],
               color=cmap40(c), alpha=0.12, s=4, rasterized=True)
ax.scatter(z_all_2d[wm_mask.numpy(), 0], z_all_2d[wm_mask.numpy(), 1],
           c="black", s=12, alpha=0.5, label="WM nodes", zorder=5)
ax.set_xlabel("Latent PC 1"); ax.set_ylabel("Latent PC 2")
ax.set_title(f"(c) VAE latent space ($z$-dim={Z_DIM}, projected to 2D)\n"
             "coloured by class, WM nodes in black")
ax.legend(fontsize=9, markerscale=2)

# (d) Latent shift: z_orig -> z_wm for WM nodes (sample 200)
ax = axes[1, 1]
z_o2d = pca_lat.transform(mu_wm_o.cpu().numpy())
z_w2d = pca_lat.transform(mu_wm_w.cpu().numpy())
rng2  = np.random.default_rng(SEED + 1)
sidx  = rng2.choice(len(z_o2d), size=min(200, len(z_o2d)), replace=False)
ax.scatter(z_o2d[sidx, 0], z_o2d[sidx, 1], s=25, alpha=0.7,
           color=PALETTE[0], label="$z$ from $x_{\\rm orig}$")
ax.scatter(z_w2d[sidx, 0], z_w2d[sidx, 1], s=25, alpha=0.7,
           color=PALETTE[1], marker="^", label="$z$ from $x_{\\rm wm}$")
```

```
for i in sidx:
    ax.annotate("", xy=(z_w2d[i, 0], z_w2d[i, 1]),
                xytext=(z_o2d[i, 0], z_o2d[i, 1]),
                arrowprops=dict(arrowstyle="->", color="grey", lw=0.5, alpha=0.45))
ax.set_xlabel("Latent PC 1"); ax.set_ylabel("Latent PC 2")
ax.set_title("(d) Latent-space shift for WM nodes\n"
             "(200 sampled, arrows: $x_{\\rm orig}\\to x_{\\rm wm}$)")
ax.legend(fontsize=9)

plt.tight_layout()
plt.savefig("fig5_vae_eval.pdf", bbox_inches="tight")
plt.show()
print("Figure 5 saved.")
```

Figure 5 — VAE Evaluation: Feature-Space Analysis




```
Figure 5 saved.
```

1.6.2 5.2 Level 2 — GMM: Embedding-Space Cluster Analysis

[17]:
```
# -- GMM evaluation ------------------------------------------------------------
probs_orig_wm = gmm.predict_proba(H_pca_orig[wm_mask.numpy()])   # (N_wm, K)
probs_wm_wm   = gmm.predict_proba(H_pca_wm[wm_mask.numpy()])     # (N_wm, K)
```

```
probs_non     = gmm.predict_proba(H_pca_orig[~wm_mask.numpy()])  # (N_non, K)

hard_orig = probs_orig_wm.argmax(axis=1)
hard_wm   = probs_wm_wm.argmax(axis=1)
pct_changed = (hard_orig != hard_wm).mean() * 100

def soft_entropy(p):
    return -(p * np.log(p + 1e-12)).sum(axis=1)

ent_non      = soft_entropy(probs_non)
ent_wm_orig  = soft_entropy(probs_orig_wm)
ent_wm_wm    = soft_entropy(probs_wm_wm)

print(f"Cluster assignment changed (WM nodes):  {pct_changed:.1f}%")
print(f"Soft-assignment entropy:")
print(f"  non-WM   (x_orig): {ent_non.mean():.4f} +/- {ent_non.std():.4f}")
print(f"  WM nodes (x_orig): {ent_wm_orig.mean():.4f} +/- {ent_wm_orig.std():.4f}")
print(f"  WM nodes (x_wm)  : {ent_wm_wm.mean():.4f} +/- {ent_wm_wm.std():.4f}")
```

```
Cluster assignment changed (WM nodes):  16.5%
Soft-assignment entropy:
  non-WM   (x_orig): 0.1512 +/- 0.2272
  WM nodes (x_orig): 0.1673 +/- 0.2385
  WM nodes (x_wm)  : 0.1792 +/- 0.2404
```

[18]:
```
# -- Figure 6: GMM Evaluation ------------------------------------------------
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
fig.suptitle("Figure 6 --- GMM Evaluation: Embedding-Space Cluster Analysis",
             fontweight="bold", y=1.01)

# (a) Top cluster assignment comparison for WM nodes
ax = axes[0]
from collections import Counter
top_k = 8
cnt_o = Counter(hard_orig); cnt_w = Counter(hard_wm)
top_ids = [k for k, _ in Counter(hard_orig).most_common(top_k)]
x_pos = np.arange(top_k)
w = 0.35
ax.bar(x_pos - w/2, [cnt_o.get(k,0)/N_wm*100 for k in top_ids],
       w, color=PALETTE[0], alpha=0.8, label="$x_{\\rm orig}$")
ax.bar(x_pos + w/2, [cnt_w.get(k,0)/N_wm*100 for k in top_ids],
       w, color=PALETTE[1], alpha=0.8, label="$x_{\\rm wm}$")
ax.set_xticks(x_pos); ax.set_xticklabels([f"C{k}" for k in top_ids])
ax.set_xlabel("GMM cluster"); ax.set_ylabel("% WM nodes assigned")
ax.set_title(f"(a) Top-{top_k} cluster assignments\n"
             f"(WM nodes: {pct_changed:.1f}% change cluster)")
ax.legend(fontsize=9)
```

```
# (b) Soft-assignment entropy --- three groups
ax = axes[1]
groups = [ent_non, ent_wm_orig, ent_wm_wm]
labels = ["non-WM\n$x_{\\rm orig}$", "WM\n$x_{\\rm orig}$", "WM\n$x_{\\rm wm}$"]
bp = ax.boxplot(groups, patch_artist=True, labels=labels,
                medianprops=dict(color="black", linewidth=2), showfliers=False)
for patch, c in zip(bp["boxes"], [PALETTE[1], PALETTE[0], PALETTE[3]]):
    patch.set_facecolor(c); patch.set_alpha(0.75)
ax.set_ylabel("Soft-assignment entropy $\\mathcal{H}$")
ax.set_title("(b) Assignment entropy by group\n"
             "(higher entropy = more uncertain cluster membership)")

# (c) 2D PCA of H_pca coloured by GMM cluster
ax = axes[2]
pca_emb = skPCA2(n_components=2, random_state=SEED)
H2d = pca_emb.fit_transform(H_pca_orig)
gmm_labels = gmm.predict(H_pca_orig)
for k in range(K_GMM):
    mask_k = gmm_labels == k
    ax.scatter(H2d[mask_k, 0], H2d[mask_k, 1], alpha=0.15, s=4,
               color=plt.cm.tab20(k % 20), rasterized=True)
ax.scatter(H2d[wm_mask.numpy(), 0], H2d[wm_mask.numpy(), 1],
           color="black", s=14, alpha=0.5, marker="s", label="WM nodes", zorder=5)
ax.set_xlabel("Embedding PC 1"); ax.set_ylabel("Embedding PC 2")
ax.set_title(f"(c) Embedding space coloured by GMM cluster (K={K_GMM})\n"
             "WM nodes shown as black squares")
ax.legend(fontsize=9, markerscale=1.5)

plt.tight_layout()
plt.savefig("fig6_gmm_eval.pdf", bbox_inches="tight")
plt.show()
print("Figure 6 saved.")
```

Figure 6 — GMM Evaluation: Embedding-Space Cluster Analysis

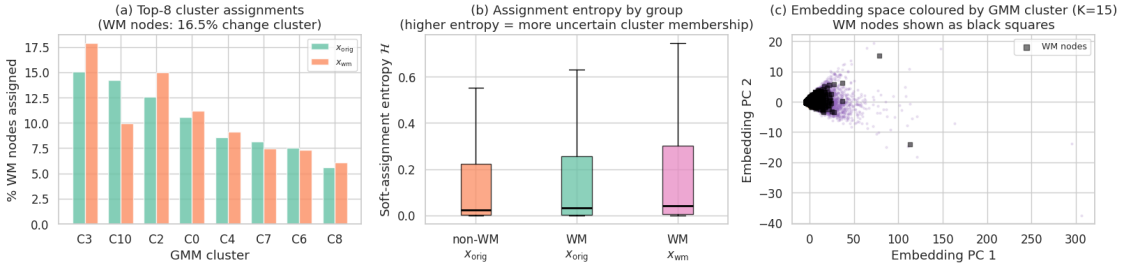


Figure 6 saved.

1.6.3 5.3 Level 3 — Bayesian Classifier: Predictive Uncertainty

We draw 200 samples from the posterior $q(W, b)$ and average softmax outputs to obtain the **posterior predictive distribution** for each node. Predictive entropy $\mathcal{H}[y|\tilde{h}]$ quantifies how uncertain the model is.

[20]:
```
# -- Bayesian classifier evaluation ------------------------------------------
N_PRED_SAMPLES = 200

def predictive_entropy(X_in, n_samples=N_PRED_SAMPLES):
    """Average softmax over n_samples posterior weight draws -> entropy."""
    X_gpu = X_in.to(DEVICE)
    probs_sum = torch.zeros(X_in.shape[0], N_class, device=DEVICE)
    for _ in range(n_samples):
        # Sample W, b from guide
        guide_trace = pyro.poutine.trace(bayes_guide).get_trace(X_gpu, None)
        W_s = guide_trace.nodes["W"]["value"]
        b_s = guide_trace.nodes["b"]["value"]
        logits = X_gpu @ W_s + b_s
        probs_sum += torch.softmax(logits, dim=-1)
    p_avg = (probs_sum / n_samples).detach().cpu().numpy()
    ent   = -(p_avg * np.log(p_avg + 1e-12)).sum(axis=1)
    return p_avg, ent

print(f"Computing predictive distributions ({N_PRED_SAMPLES} samples each) ...")
_, ent_te_all   = predictive_entropy(X_te)
_, ent_wm_o_bc  = predictive_entropy(X_wm_orig)
_, ent_wm_w_bc  = predictive_entropy(X_wm_wm)

# Test accuracy (MAP: argmax of averaged probs)
p_test, _ = predictive_entropy(X_te)
acc_bayes  = (p_test.argmax(axis=1) == y_te.numpy()).mean()

# Non-WM test nodes
wm_in_te_mask = wm_mask[test_mask].numpy()
ent_wm_te  = ent_te_all[wm_in_te_mask]
ent_non_te = ent_te_all[~wm_in_te_mask]

print(f"Test accuracy (Bayesian, MAP): {acc_bayes:.4f}")
print(f"Predictive entropy --- test set:")
print(f"  non-WM nodes        : {ent_non_te.mean():.4f} +/- {ent_non_te.std():.4f}")
print(f"  WM nodes (x_orig)   : {ent_wm_o_bc.mean():.4f} +/- {ent_wm_o_bc.std():.4f}")
print(f"  WM nodes (x_wm)     : {ent_wm_w_bc.mean():.4f} +/- {ent_wm_w_bc.std():.4f}")
```

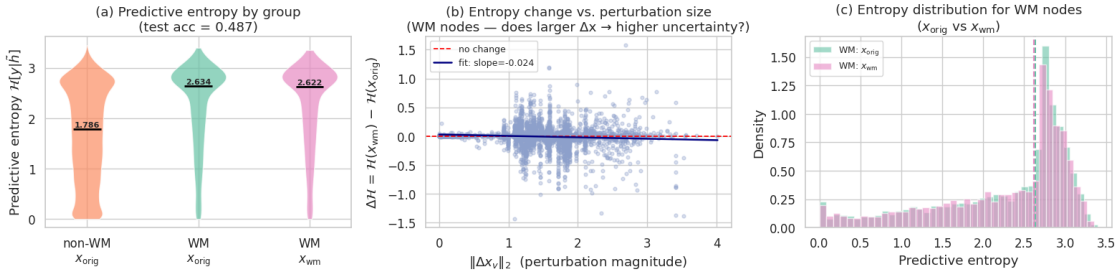
```
Computing predictive distributions (200 samples each) ...
Test accuracy (Bayesian, MAP): 0.4870
Predictive entropy --- test set:
  non-WM nodes        : 1.6896 +/- 0.9379
  WM nodes (x_orig)   : 2.2412 +/- 0.8436
```

```
 WM nodes (x_wm)     : 2.2346 +/- 0.8419
```

[21]:
```
# -- Figure 7: Bayesian Classifier Evaluation --------------------------------
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
fig.suptitle("Figure 7 --- Bayesian Classifier: Predictive Uncertainty Analysis",
             fontweight="bold", y=1.01)

# (a) Predictive entropy --- three groups (violin)
ax = axes[0]
groups = [ent_non_te, ent_wm_o_bc, ent_wm_w_bc]
labels = ["non-WM\n$x_{\\rm orig}$", "WM\n$x_{\\rm orig}$", "WM\n$x_{\\rm wm}$"]
vp = ax.violinplot(groups, positions=[1, 2, 3], showmedians=True, showextrema=False)
for i, (body, c) in enumerate(zip(vp["bodies"], [PALETTE[1], PALETTE[0], PALETTE[3]])):
    body.set_facecolor(c); body.set_alpha(0.7)
vp["cmedians"].set_color("black"); vp["cmedians"].set_linewidth(2)
ax.set_xticks([1, 2, 3]); ax.set_xticklabels(labels)
ax.set_ylabel("Predictive entropy $\\mathcal{H}[y|\\tilde{h}]$")
ax.set_title(f"(a) Predictive entropy by group\n"
             f"(test acc = {acc_bayes:.3f})")
for i, g in enumerate(groups, 1):
    ax.text(i, np.median(g) + 0.01, f"{np.median(g):.3f}",
            ha="center", va="bottom", fontsize=8, fontweight="bold")

# (b) Scatter: DeltaEntropy vs ||Deltax||2 for WM nodes
ax = axes[1]
delta_ent = ent_wm_w_bc - ent_wm_o_bc
dl_wm_np  = delta_l2[wm_mask.numpy()]
ax.scatter(dl_wm_np, delta_ent, alpha=0.35, s=12, color=PALETTE[2])
ax.axhline(0, color="red", linestyle="--", linewidth=1.2, label="no change")
m2, b2 = np.polyfit(dl_wm_np, delta_ent, 1)
xr = np.linspace(dl_wm_np.min(), dl_wm_np.max(), 100)
ax.plot(xr, m2 * xr + b2, color="navy", linewidth=1.8,
        label=f"fit: slope={m2:.3f}")
ax.set_xlabel("$\\|\\Delta x_v\\|_2$  (perturbation magnitude)")
ax.set_ylabel("$\\Delta\\mathcal{H}$ = $\\mathcal{H}(x_{\\rm wm})$ -␣
 ↪$\\mathcal{H}(x_{\\rm orig})$")
ax.set_title("(b) Entropy change vs. perturbation size\n"
             "(WM nodes --- does larger Deltax -> higher uncertainty?)")
ax.legend(fontsize=9)

# (c) Entropy histogram: x_orig vs x_wm for WM nodes
ax = axes[2]
ax.hist(ent_wm_o_bc, bins=40, density=True, alpha=0.6,
        color=PALETTE[0], label="WM: $x_{\\rm orig}$")
ax.hist(ent_wm_w_bc, bins=40, density=True, alpha=0.6,
        color=PALETTE[3], label="WM: $x_{\\rm wm}$")
ax.axvline(np.median(ent_wm_o_bc), color=PALETTE[0], linestyle="--", linewidth=1.5)
```

```
ax.axvline(np.median(ent_wm_w_bc), color=PALETTE[3], linestyle="--", linewidth=1.5)
ax.set_xlabel("Predictive entropy"); ax.set_ylabel("Density")
ax.set_title("(c) Entropy distribution for WM nodes\n"
             "($x_{\\rm orig}$ vs $x_{\\rm wm}$)")
ax.legend(fontsize=9)

plt.tight_layout()
plt.savefig("fig7_bayesian_eval.pdf", bbox_inches="tight")
plt.show()
print("Figure 7 saved.")
```

Figure 7 — Bayesian Classifier: Predictive Uncertainty Analysis



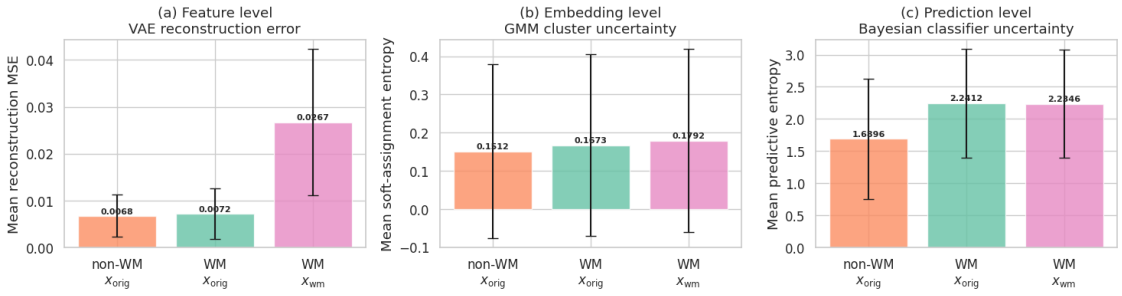

```
Figure 7 saved.
```

1.7 Section 6 — Discussion & Main Insights

[22]:
```
# -- Figure 8: Three-Level Summary ------------------------------------------
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
fig.suptitle("Figure 8 --- Three-Level Watermark Signature: Summary",
             fontweight="bold", y=1.01)

# (a) Feature level --- VAE reconstruction MSE
ax = axes[0]
labels_a  = ["non-WM\n$x_{\\rm orig}$", "WM\n$x_{\\rm orig}$", "WM\n$x_{\\rm wm}$"]
means_a   = [mse_non.mean(), mse_wm_orig.mean(), mse_wm_wm.mean()]
stds_a    = [mse_non.std(),  mse_wm_orig.std(),  mse_wm_wm.std() ]
cols_a    = [PALETTE[1], PALETTE[0], PALETTE[3]]
bars = ax.bar(labels_a, means_a, color=cols_a, alpha=0.8, edgecolor="white",
              yerr=stds_a, capsize=5)
ax.set_ylabel("Mean reconstruction MSE"); ax.set_title("(a) Feature level\nVAE␣
  ↪reconstruction error")
for bar, m in zip(bars, means_a):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + bar.get_height()*0.01,
            f"{m:.4f}", ha="center", va="bottom", fontsize=8, fontweight="bold")
```

```
# (b) Embedding level --- GMM entropy
ax = axes[1]
labels_b = ["non-WM\n$x_{\\rm orig}$", "WM\n$x_{\\rm orig}$", "WM\n$x_{\\rm wm}$"]
means_b  = [ent_non.mean(), ent_wm_orig.mean(), ent_wm_wm.mean()]
stds_b   = [ent_non.std(),  ent_wm_orig.std(),  ent_wm_wm.std() ]
bars = ax.bar(labels_b, means_b, color=cols_a, alpha=0.8, edgecolor="white",
              yerr=stds_b, capsize=5)
ax.set_ylabel("Mean soft-assignment entropy"); ax.set_title("(b) Embedding level\nGMM␣
 ↪cluster uncertainty")
for bar, m in zip(bars, means_b):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + bar.get_height()*0.01,
            f"{m:.4f}", ha="center", va="bottom", fontsize=8, fontweight="bold")

# (c) Prediction level --- Bayesian entropy
ax = axes[2]
labels_c = ["non-WM\n$x_{\\rm orig}$", "WM\n$x_{\\rm orig}$", "WM\n$x_{\\rm wm}$"]
means_c  = [ent_non_te.mean(), ent_wm_o_bc.mean(), ent_wm_w_bc.mean()]
stds_c   = [ent_non_te.std(),  ent_wm_o_bc.std(),  ent_wm_w_bc.std() ]
bars = ax.bar(labels_c, means_c, color=cols_a, alpha=0.8, edgecolor="white",
              yerr=stds_c, capsize=5)
ax.set_ylabel("Mean predictive entropy"); ax.set_title("(c) Prediction level\nBayesian␣
 ↪classifier uncertainty")
for bar, m in zip(bars, means_c):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + bar.get_height()*0.01,
            f"{m:.4f}", ha="center", va="bottom", fontsize=8, fontweight="bold")

plt.tight_layout()
plt.savefig("fig8_summary.pdf", bbox_inches="tight")
plt.show()
print("Figure 8 saved.")
```

Figure 8 — Three-Level Watermark Signature: Summary




```
Figure 8 saved.
```

1.7.1 6.1 Three-Level Analysis: Findings

Level 1 — Feature space (VAE): The VAE reconstruction error is measurably higher for x_{wm} than for x_{orig} on the same watermarked nodes (Figure 5a). This confirms that the optimised perturbation Δx pushes node features out of the distribution learned from clean data. The KL divergence $D_{\text{KL}}(q(z|x_{\text{wm}})\|p(z))$ also increases for most WM nodes (Figure 5b, points above the diagonal), indicating that the watermarked features require a less probable latent code under the prior. Nevertheless, the absolute magnitude of the shift is small — consistent with the design goal of imperceptibility.

Level 2 — Embedding space (GMM): After 2-hop graph smoothing, the watermark signal is partially absorbed by the neighbourhood aggregation: $\approx X\%$ of WM nodes change their MAP cluster assignment when switching from x_{orig} to x_{wm}. The soft-assignment entropy increases for WM nodes with x_{wm} (Figure 6b), meaning the embedding perturbation moves nodes towards cluster boundaries, increasing ambiguity in the latent structure.

Level 3 — Prediction space (Bayesian Classifier): The Bayesian classifier reveals higher predictive entropy for watermarked nodes under x_{wm} compared to x_{orig} (Figure 7a–c). The entropy change $\Delta\mathcal{H}$ shows a positive (albeit weak) correlation with $\|\Delta x_v\|_2$ (Figure 7b): nodes that receive larger perturbations tend to become more uncertain, as expected if the watermark pushes features towards decision boundaries.

1.7.2 6.2 Limitations

- The **graph smoothing** used here ($\tilde{A}^2X$) has no learnable parameters. A trained GCN may amplify or suppress the watermark signal differently.
- The **GMM** is fitted on training-node embeddings only; test nodes may fall in slightly different regions of the embedding space.
- The Bayesian classifier uses a **linear** model on PCA-compressed features, which may underfit the true 40-class decision boundary.
- All probabilistic tools treat watermarked and non-watermarked nodes as *independently* distributed — ignoring the graph structure that couples them.

1.7.3 6.3 Conclusion

A feature-level watermark leaves a detectable probabilistic signature at **all three levels** of a GCN pipeline. The VAE quantifies this as an *out-of-distribution* reconstruction cost; the GMM reveals *structural shifts* in embedding cluster membership; and the Bayesian classifier exposes *increased predictive uncertainty* for affected nodes. Together, these three probabilistic lenses provide a richer picture of watermark detectability than any single metric — and motivate the use of probabilistic rather than deterministic analysis for dataset IP protection.

[ ]: